Assignment 6: Formal Grammar and Ambiguity

CAS CS 320: Concepts of Programming Languages

due Thursday, October 22 by 8:00PM

There are no OCaml problems this week.

For the written problems, you'll submit your solutions via Gradescope as a single PDF file. They must be hand-written and exceptionally neat, and the final answer to each problem must be clear, e.g., surrounded by a box. You must cite your sources per the instructions in the course manual.

You're required to submit solutions to every problem in this assignment to receive full credit, but we will only provide detailed feedback on **bolded** problems. Since the assignment is graded on participation, we ask that you *try at least the bolded problems with as few external aids as possible.* You should think of these problems as simulated quizzes or exams. *You won't be penalized for mistakes*, so it's in your best interest to do these problems on your own; the detailed feedback we provide will generally be more useful (for you and for us) if you submit your own work.

Written Problems

1.

```
<s> ::= A <s> B | <a>
<a> ::= A <a> | A <b>
<b> ::= B B <b> | B B
```

(a) Demonstrate that the sentence `A A A B B B` is recognized by the above grammar by providing a parse tree for it.

(b) Demonstrate that the above grammar is ambiguous by determining the *shortest* sentence which has multiple parse trees. Include both parse trees in your solution.

(2)

```
<e>  ::= match <e> with <cs> | x | true | false | (<e>)
<cs> ::= <c> | <c> <cs>
<c>  ::= | <p> -> <e>
<p>  ::= x | true | false
```

(a) Demonstrate that the sentence `match x with | true -> false | false -> true` is recognized by the above grammar by providing a parse tree for it.

(b) Demonstrate that the above grammar is ambiguous by determining a sentence which has multiple parse trees. Include both parse trees in your solution.

3.

```
<e>  ::= if <e> then <e> else <e> | <e> + <e2> | <e2>
<e2> ::= true | false | 1 | ( <e> )
```

(a) Demonstrate that the sentence `if true then 1 + 1 else 1` is recognized by the above grammar by providing a parse tree for it.

(b) Demonstrate that the above grammar is ambiguous by determining a sentence which has multiple parse trees. Include both parse trees in your solution.

(4) The following grammar expresses that a program (<p>) is a sequence of assignment statements (<w> = <e>) ending with an optional expression (<e>).[1] For example, x = 1 y = & x x + * y is a well-formed program. Note that it can be written more naturally as:[2]

```
x = 1
y = &x
x + *y
```

<p> ::= <w> = <e> <p> | <w> = <e> | <e>
<e> ::= <e2> <o> <e> | <e2>
<e2> ::= & <w> | <w> | (<e>) | 1 | 2 | 3
<o> ::= + | - | * | /
<w> ::= * <w> | x | y | z

(a) Demonstrate that the sentence x = 1 y = & x x + * y is recognized by the above grammar by providing a parse tree for it.

(b) Demonstrate that the above grammar is ambiguous by determining a sentence which has multiple parse trees. Include both parse trees in your solution.

[1] This is similar to how Rust works.

[2] The meaning of this program doesn't matter for this problem, but we might understand this program to be saying: `x` is assigned to `1`, and then `y` is assigned to a reference to `x`, and then the output of the program is `x` plus the value that `y` refers to, i.e., `y` is *dereferenced*.